

목차 — Table of Contents

이 리포트는 표지 → 목차 → 11개 챕터 순으로 구성된다. 아래 항목을 클릭(또는 PDF 뷰어의 outline 패널) 하여 해당 챕터로 점프할 수 있다. 각 챕터 머리에는 본문 라벨이 붙은 그림과 표가 배치되어 있다.

- [00Overview — Paperclip 은 무엇인가정체성 · 3-zone · 5대 원칙](#)
- [01Architecture — 모노레포 구조pnpm workspace · 8 패키지 · dev 명령](#)
- [02Data Model — 75+ 테이블 한 번에 보기핵심 ER · atomic checkout · 도메인 군집](#)
- [03Runtime Execution — Heartbeat · Checkout · Watchdog시퀀스 · 7-state · stranded recovery](#)
- [04Adapters & Skills — 8개 어댑터 + Paperclip Skill능력 매트릭스 · 4-surface · MCP](#)
- [05Server & API — Express · Routes · Services · WebSocketroutes · services · realtime · auth](#)
- [06UI & Board — React · Vite · shadcn/ui70+ 페이지 · TanStack Query · 디자인 가이드](#)
- [07Governance & Cost — Board · Approvals · Budget · Activity Log승인 · 예산 6단계 · 감사](#)
- [08Usage — 설치 · 회사 만들기 · 첫 운영5분 시작 · CEO 첫 에이전트 · 운영 점검](#)
- [09Pros & Cons — Paperclip 의 트레이드오프11 강점 · 10 약점 · 7 트레이드오프](#)

그림과 표 인덱스

본문에는 자체 SVG 10장 + 외부 인용 그림 17장 + 표 21개 가 배치되어 있다. 모두 동일한 캡션 형식 — ****그림 N. <제목>**
**** / **표 N. <제목> **** — 으로 본문 라벨이 붙는다.

종류	위치	라벨
자체 그림	00\~10 챕터	그림 1 / 1-A · 2 · 3 · 4 · 4-A · 4-B · 5 · 5-A · 6 · 6-1 · 7 · 4-1\~4 · 8 · 9 · 10 · 10-1\~9
표	00\~10 챕터	표 1\~3 (각 챕터별 1\~3개)

그림과 표의 읽는 법 은 본문 단락에 함께 주석되어 있다 — 각 그림/표가 어느 방향으로 읽는지, 어느 디테일을 주목해야 하는지, 다른 챕터의 어느 부분과 연결되는지 1\~2문단 분량으로 보강했다.

시각 스타일 규칙

- 잉크 색 #1a1a2e , 액센트 레드 #c84a3a , 네이비 #0f2c59 , 회색 톤 #5b5b62 .
- 표·그림 번호는 본문에서 ****그림 N. 〈제목〉**** · ****표 N. 〈제목〉**** 형식으로 부여한다(외부 이미지는 alt-text 에 출처 동행).
- SVG 본체에는 제목/캡션을 넣지 않는다(다이어그램 내부 데이터 라벨은 허용).
- 한국어 본문 + 영어 고유명사 병기. CJK bold 규칙(****용어(English)**** 는) 준수.

라이선스 / 저작권 안내

이 리포트는 학습 목적의 2차 정리물이다. 인용한 외부 이미지는 모두 출처 페이지로 링크되며 원 저작자의 권리를 그대로 둔다. 인용 부적절성이 발견되면 [figs/external/MANIFEST.md](#) 의 해당 항목과 본문 인용을 제거하면 된다.

Overview — Paperclip은 무엇인가

1. 한 줄 요약

Paperclip은 AI 에이전트들로 구성된 회사를 운영하기 위한 **오픈소스 control plane**이다. “회사” 단위로 에이전트들이 조직되고, 작업이 단일 골(goal)을 향해 위계적으로 분해되며, 사람(보드 운영자)은 승인·예산·일시정지 권한으로 거버넌스를 행사한다.

저장소의 슬로건은 핵심을 압축한다 — “If OpenClaw is an *employee*, Paperclip is the *company*.” 즉 Paperclip은 **에이전트 자체가 아니라, 에이전트를 고용·조직·예산 통제·감사하는 회사 운영체제**다. 코어 서버는 LLM 호출/툴 사용 같은 *에이전트 런타임 로직*을 내장하지 않으며, 그 역할은 어댑터(외부 CLI·웹훅·원격 게이트웨이)가 맡는다.

2. 풀려는 문제

저장소 README([README.md:131-140](#))가 “Problems Paperclip solves” 표로 든 출발점은 여섯 가지다.

1. Claude Code 같은 코딩 에이전트 터미널을 20개 열어 두면 어떤 탭이 무엇을 하는지 추적할 수 없고, 재부팅하면 다 잃는다.
2. 에이전트가 무엇을 왜 해야 하는지 매번 컨텍스트를 떠먹여 줘야 한다.
3. 에이전트 설정 폴더가 흩어져 있으면 작업 할당·커뮤니케이션·조정을 다시 발명하게 된다.
4. 비제한 루프가 토큰을 폭주 소비하면 사후에야 알게 된다.
5. 정기적인 작업(고객 지원, 마케팅, 리포트)을 매번 수동으로 켜야 한다.
6. 아이디어가 떠올라도 repo를 찾고 Claude Code를 띄우고 탭을 열어 둔 채 babysit해야 한다.

Paperclip의 제안은 위 문제를 **작업 관리자 형태의 control plane**으로 통합해 푸는 것이다. 사용자는 “골”을 정의하고, 에이전트를 “고용”하며, 보드(인간 운영자)는 모니터링·승인·일시정지 권한으로 개입한다.

3. 큰 그림

그림 1. Paperclip의 3-zone 큰 그림 은 시스템이 항상 세 영역으로 분리되어 있음을 보여 준다. 왼쪽은 **인간 보드(Board)** 영역으로, 운영자는 React 보드 UI · CLI · 모바일 웹을 통해 들어온다. 가운데가 Paperclip 본체 — REST API, 오케스트레이션 서비스, Drizzle로 관리되는 PostgreSQL, MCP 서버, 플러그인 런타임이 모인 **control plane**이다. 오른쪽은 **execution plane**으로, 현재 upstream `master` 기준 13개의 built-in adapter type(`server/src/adapters/registry.ts:511-524`)이 외부 에이전트 런타임을 호출한다. 11개는 구체 런타임(Claude Code, Codex, Cursor local, Cursor Cloud, Gemini, Grok local, OpenCode, Pi, ACP-x, OpenClaw 게이트웨이, Hermes)을 향하고, 2개는 범용 `process` / `http` 백업 통로다. Paperclip은 *에이전트 런타임 로직*을 내장하지 않는다 — heartbeat를 보내 child process 또는 webhook으로 외부 런타임을 깨우고, 결과를 거두어 비용 · 상태를 추적한다.

그림 1. Paperclip의 3-zone 큰 그림 — Board · Control plane · Execution plane

그림의 핵심은 **세 영역 사이의 화살표 방향**이다. 보드 → control plane은 *인증된 REST 요청* 한 가지로 단순화되어 있고, control plane → execution plane은 *어댑터 계약*(`execute / testEnvironment / ...`)으로 추상화되어 있다. 즉 좌우⁸ 두

4. 5개 핵심 원칙

PRODUCT.md(doc/PRODUCT.md:61-71)의 다섯 제품 원칙과 SPEC.md § 13(doc/SPEC.md:521-531)의 아홉 consolidated principles를 학습용 다섯 축으로 추리면 다음과 같다 — 원전 두 문서가 완전히 동일한 5개를 강조한다는 뜻은 아니다.

표 1 Paperclip의 5대 설계 원칙

#	원칙	무엇을 의미하는가
1	Unopinionated about how you run agents	Python 스크립트, Claude Code, OpenClaw 봇 — 어떤 런타임이든 어댑터로 들어온다.
2	Company is the unit of organization	모든 데이터 엔티티가 company_id 로 스코프된다. 한 인스턴스에 여러 회사를 둘 수 있다.
3	Tasks are the communication channel	별도의 채팅·메신저 시스템 없음. 모든 통신은 issue + 댓글로 흘러 자연스러운 감사 이력이 된다.
4	All work traces to the goal	원칙: 모든 issue는 부모 사슬을 따라 회사 골(혹은 initiative)에 도달해야 한다. 구현: goal_id / parent_id 가 nullable이며 누락 시 fallback이 회사 기본 goal로 채우려 시도한다

5. V1 범위와 범위 밖 항목

`doc/SPEC-implementation.md:60-82` 이 정의한 V1 범위 안과 범위 밖 항목은 다음과 같다.

표 2 V1 스코프 요약

분류	항목
In	회사 CRUD, 골 위계, 어댑터·org chart가 포함된 에이전트 lifecycle, parent/child issue, atomic checkout, 보드 승인(고용·CEO 전략), heartbeat, cost ingestion + rollup, 예산 hard-stop, 보드 웹 UI(대시보드·org chart·issues·승인·비용), 에이전트 REST API, 활동 로그
Out	클라우드 플러그인 마켓플레이스(self-hosted plugin runtime만 범위 안), 토큰 외부의 매출/비용 회계, 지식 베이스(KB) 서브시스템, 공개 마켓플레이스(ClipHub), 멀티-보드 거버넌스, 자동 자가 치유 오케스트레이션

표 2에서 두 가지가 두드러진다. 첫째, **Out 항목의 성격은 균질하지 않다** — KB는 future plugin으로 명시되어 있고 (`doc/SPEC.md:491`, `:500-502`), 외부 매출/비용 회계도 future plugin이다(`doc/SPEC.md:493`, `:517`). 반면 공개 마켓플레이스(ClipHub)는 future plugin이 아니라 V1 밖에 머무는 항목이고(`doc/SPEC-implementation.md:80`), multi-board governance와 self-healing orchestration은 V1 밖의 거버넌스/오케스트레이션 확장으로 남아 있다 (`doc/SPEC-implementation.md:81-82`). 즉 Out 열을 통째로 “플러그인 영역”이라고 부르면 부정확하다. 둘째, 자동

6. 누가 쓰면 좋은가

README의 “Paperclip is right for you if” 목록([README.md:70-78](#))은 일곱 가지 사용 조건을 든다.

- **자율 AI 회사 빌더:** 한 인간이 여러 에이전트 회사를 동시 운영하려는 경우.
- **다중 에이전트 조정자:** OpenClaw + Codex + Claude + Cursor를 섞어 쓰면서 공통 골에 정렬시키고 싶은 경우.
- **20-탭 Claude Code 운영자:** 누가 무엇을 하는지 잃어버리는 사람.
- **24/7 운영을 원하지만 감사·개입은 유지하려는 사람.**
- **비용 모니터링·예산 집행이 필요한 사람:** 토큰 비용이 한 번 폭주하면 알아채기 늦는 환경.
- **태스크 매니저 같은 흐름으로 에이전트를 관리하려는 사람:** 트렐로/리니어를 쓰던 감각으로 들어오는 운영자.
- **모바일에서 자율 사업체를 모니터링하려는 사람.**

[01-architecture.md](#)는 이 큰 그림을 코드 모노레포 구조로 확장해 분석한다.

Architecture — 모노레포 구조

1. 모노레포 한눈에

Paperclip은 **pnpm workspace** 모노레포다. 루트의 `pnpm-workspace.yaml` 이 워크스페이스를 선언하고, 각 패키지는 `workspace:*` 프로토콜로 서로를 참조한다. 그림 1-1은 패키지 지도와 의존 방향을 한 페이지로 보여 준다. 의존 화살표는 “위쪽 패키지가 아래쪽 패키지를 import 한다”를 의미한다.

그림 1-1. pnpm 모노레포 패키지 지도와 의존 방향

그림 1-1의 구조는 세 계층으로 나뉜다. (1) 최상위 **애플리케이션** — `server/`, `ui/`, `cli/`. (2) **공유 코어** — `packages/db`, `packages/shared`, `packages/mcp-server`. (3) **확장점** — `packages/adapters/*`, `packages/adapter-utils`, `packages/plugins`. 외부에는 사용자가 설치하는 **어댑터 플러그인**이 점선으로 그려져 있는데, 이는 코어를 수정하지 않고 새로운 에이전트 런타임을 끼워 넣는 통로다. 의존이 위에서 아래로만 흐른다는 점도 그림에서 확인할 수 있다 — `packages/shared` 가 `server/` 를 import 하지 않듯, 하위 패키지는 상위 패키지를 참조하지 않으므로 모노레포 빌드를 위상 정렬할 수 있고, 하위에서 발생한 타입 변경이 상위로 전파된다.

2. 패키지별 책임 — 표 한 장

표 1 패키지별 역할과 주요 런타임 의존

패키지	역할	주요 export	의존
server/	Express REST API + 오케스트레이션 호스트	app.ts , routes/* , services/* , realtime/*	db · shared · adapter-utils · plugin-sdk · 9개 adapter workspace · hermes-paperclip-adapter (npm) (server/package.json:47-70)
ui/	React + Vite 보드 UI	페이지·컴포넌트·api 클라이언트	shared · 9개 adapter workspace · adapter-utils · hermes-paperclip-adapter (ui/package.json:37-54) — 어댑터 UI entry 동적 임포트용
cli/	paperclipai 단일 진입 CLI	src/index.ts (commands: onboard · doctor · run · plugin · db:backup)	shared · server · db · adapter-utils · 9개 adapter workspace (cli/package.json:40-57) — in-process boot

3. built-in 어댑터 타입과 패키지형 어댑터

현재 upstream `master` 의 built-in adapter type은 13개다(`server/src/adapters/registry.ts:511-524`).
패키지형 10종 — `acpx-local`, `claude-local`, `codex-local`, `cursor-cloud`, `cursor-local`, `gemini-local`,
`grok-local`, `opencode-local`, `pi-local`, `openclaw-gateway` — 은 `packages/adapters/` 아래의 워크스페이스
패키지(`server/package.json:47-55`)로 들어 있고, `hermes_local` 은 `hermes-paperclip-adapter` npm 의존성을
`server/src/adapters/registry.ts:128-138` 에서 정적으로 등록한다. 나머지 `process` 와 `http` 는 새 런타임을
임시로 붙일 때 쓰는 범용 generic 통로다. 아래 코드 1은 패키지형 어댑터 디렉터리의 표준 트리다.

코드 1. 어댑터 패키지의 표준 디렉터리 구조



이 4개 진입점(`. / server / ui / cli`)은 `package.json` 의 `exports` 필드로 외부에 노출된다. 즉 어댑터는 동일한
npm 패키지가 `server·ui·cli` 에 각자 다른 모듈을 제공 하는 식으로 짜여 있다. 이렇게 하면 서버는 실행 로직만, UI 는 설정
폼만 가져갈 수 있어 트리 셰이킹·번들 크기 면에서 유리하다. 장기 SPEC(`doc/SPEC.md:207-215`)은

AI Odyssey
`invoke/status/cancel` 3개 메서드를 최소 계약으로 그리지만, 현재 구현의

`ServerAdapterModule` (`packages/adapter_utils/src/types.ts:340-352`)이 피스르 드는 메서드는 `execute` 와

4. 단일 origin dev 모드

개발 단계에서는 Vite 의 **middleware mode** 가 핵심이다. `pnpm dev` 는 Express 서버를 띄우면서 동시에 `vite.middlewareMode` 로 SPA 자산을 같은 *origin* 에서 서빙한다. 기본 요청 포트는 `3100` 이며, 점유 중이면 `detect-port (server/package.json:64)` 가 다음 사용 가능한 포트를 골라 그쪽으로 올린다 — UI 도 선택된 API 포트에 same-origin 으로 따라간다. 이 결과 dev 모드에서는 CORS 설정이 필요 없고, 보드 UI 의 `fetch('/api/...')` 가 그대로 같은 origin 으로 떨어진다. 코드 2 가 dev 워크플로의 4가지 모드를 한 줄씩 보여 준다 — 일상은 `pnpm dev` 하나로 충분하지만, 서버/UI 를 따로 디버깅해야 할 때를 위해 분리 모드도 준비되어 있다.

코드 2. 개발 워크플로 — 4가지 dev 명령



또 하나의 트릭은 `scripts/dev-runner.ts` 다. 같은 저장소·같은 인스턴스에 대해 `pnpm dev` 를 두 번째로 호출해도 기존 dev 러너를 재사용해 중복 실행을 막는다. `pnpm dev:list / dev:stop` 으로 관리할 수 있다.

5. 빌드·테스트 토폴로지

빌드의 표준 진입점은 루트 `pnpm build` 다(`package.json:16`). 이 스크립트가 먼저 `preflight:workspace-links` 단계로 워크스페이스 심볼릭 링크 정합을 점검하고, 이어서 `pnpm -r build` 로 모든 워크스페이스를 위상 정렬해 빌드한다. `pnpm -r build` 만 직접 실행하면 `preflight` 단계는 건너뛴다는 점에 주의해야 한다. 테스트는 두 갈래다.

- **단위/통합:** Vitest 3.x — `pnpm test` (= `pnpm test:run` = `node scripts/run-vitest-stable.mjs`). `general` 모드와 `serialized` 모드가 분리되어 있다.
- **E2E / 릴리스 스모크:** Playwright — `pnpm test:e2e`, `pnpm test:release-smoke`. 변경이 UI 흐름을 건드릴 때만 켜다.

`package.json` 의 `engines.node = ">=20"` 이 강제하는 Node 20 LTS 위에서 동작한다. 표 2 가 이를 정리한다.

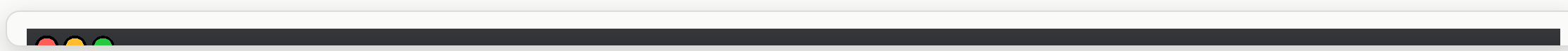
표 2 표준 명령 한눈에

명령	무엇을 하는가
<code>pnpm install</code>	의존성 + workspace 링크

6. 두 종류의 인스턴스 디렉터리

런타임 데이터는 **저장소 밖**의 `~/.paperclip/instances/<instance>/`에 산다. 코드 3은 한 인스턴스의 **핵심 경로만** 추린 것이고, 전체 레이아웃(예: `data/backups/`, `logs/`, `workspaces/<agent-id>/`, `projects/`, `companies/<company-id>/codex-home/` 등)은 `doc/DEVELOPING.md`의 인스턴스 섹션을 참고해야 한다. 코드 저장소와 사용자 데이터가 깨끗이 분리되어 있어, 같은 머신에서 여러 인스턴스를 병렬로 띄울 수 있다.

코드 3. 인스턴스 디렉터리 레이아웃 (`~/.paperclip/instances/<instance>/`)



이는 두 가지를 가능케 한다. (1) 코드 저장소가 사용자 데이터를 깨끗이 분리한다. (2) 한 머신에서 여러 인스턴스를 병렬로 운영할 수 있다 — `PAPERCLIP_HOME` 또는 `PAPERCLIP_INSTANCE_ID`로 인스턴스 디렉터리를 분리하면, 포트는 `detect-port`가 사용 가능한 값으로 알아서 조정한다.

[02-data-model.md](#)는 이 패키지 구조 위의 데이터 모델 — Drizzle 스키마 80+ 테이블 — 을 핵심 ER 그림으로 분석한다.

Data Model — 80개 이상 테이블 한눈에 보기

1. 데이터 모델 한눈 — 핵심 ER

`packages/db/src/schema/`에는 86개의 `pgTable()` 정의가 78개 스키마 파일에 분산되어 있다 (`packages/db/src/schema/index.ts:1-78`). 모두를 한 그림에 담는 것은 무의미하므로, 보드 운영 흐름을 따라가는 데 필수적인 12~13개 테이블만 추려 그림 4로 묶었다.

그림 4. 핵심 ER 다이어그램 — issues를 중심으로 한 12개 이상 테이블

가운데 빨간 박스가 **issues** — 단일 작업 단위(unit of work)다. 위쪽 회색 박스 4개(`agents` · `goals` · `projects` · `approvals`)는 모두 `companies`에서 내려온다. 아래쪽 청색 박스 그룹(`cost_events` · `budget_policies` · `budget_incidents` · `activity_log`)은 비용·거버넌스 레이어다. 이 그림은 모든 비즈니스 데이터가 `company_id`로 **스코프**되며, 이슈 한 개가 어떻게 cost·activity·heartbeat와 연결되는지를 보여 준다. 핵심 관찰점은 세 가지다. 첫째, `issues` 박스 안의 `assignee_agent_id XOR assignee_user_id` 한 줄은 한 이슈를 오직 한 명의 에이전트 또는 사람이 책임지는 단일 assignee 규칙(§ 4)의 시각적 증거다. 둘째, `issues`는 `checkout_run_id`와 `execution_run_id` 두 컬럼으로 `heartbeat_runs.id`를 직접 참조하고(`packages/db/src/schema/issues.ts:37-38`),

2. 단일 불변항: company-scoped

AGENTS.md Core Engineering Rule #1이 강제하는 단일 불변항은 명료하다 — “*Every domain entity should be scoped to a company and company boundaries must be enforced in routes/services.*” 80여 개 테이블 중 60여 개가 `company_id uuid not null references companies.id` 컬럼을 들고 있다. 라우트 레이어의 미들웨어가 모든 mutating call에 대해 회사 멤버십·권한을 검사한다.

3. 7개 핵심 테이블 — 표로 본 컬럼 요약

표 1 기동 테이블 7개의 핵심 컬럼

테이블	핵심 컬럼	무엇을 의미
companies	id, status, issue_prefix, issue_counter, budget_monthly_cents, spent_monthly_cents, require_board_approval_for_new_agents, brand_color	회사 = 모든 데이터의 루트. issue 번호도 회사별 카운터.
agents	company_id, reports_to (FK→agents), adapter_type, adapter_config jsonb, runtime_config jsonb, default_environment_id, budget_monthly_cents, last_heartbeat_at	트리 형태 org. adapter 타입과 jsonb 설정이 에이전트의 본체.
issues	company_id, parent_id (self FK), project_id, issue_number + identifier (예: PAP-123), status, assignee_agent_id ⊕ assignee_user_id, checkout_run_id, execution_run_id, execution_policy jsonb, billing_code, origin_kind/origin_id/origin_run_id	단일 이슈 모델. assignee는 한 명뿐이다. blocker는 별도 relation/ PATCH /issues/:id 의 blockedByIssueIds payload로 다룬다.

4. issue assignee의 XOR 규칙

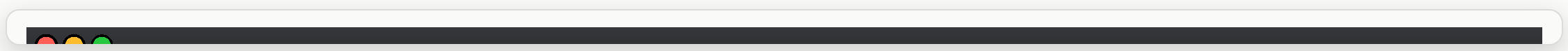
`issues`의 `assignee_agent_id`와 `assignee_user_id`에는 동시에 값이 들어갈 수 없다 — `doc/execution-semantics.md` § 2가 강한 불변조건으로 명시한다. 이유는 깔끔하다. 에이전트가 가진 이슈는 heartbeat scheduler의 실행 루프에 들어가지만, 사람이 가진 이슈는 들어가지 않는다. 한 자리에 둘이 동시에 있으면 회복 규칙을 단순하게 적을 수 없다.

이 분리 덕분에 `status='in_progress'`가 에이전트 소유면 “라이브 실행이 보장돼야 한다”는 강한 계약을 둘 수 있다. 사람 소유 이슈는 같은 status라도 heartbeat가 깨지지 않는다.

5. atomic checkout — 동시성을 데이터 모델로 푼다

이슈 한 개에 대한 동시 작업을 막기 위해, Paperclip은 낙관적 락이나 CRDT가 아닌 **atomic UPDATE**로 푼다. 핵심은 `doc/SPEC.md` § 8의 의사코드 한 줄 — “*affected_rows == 1 일 때만 성공*” — 이지만, 실제 구현은 `expectedStatuses`, `assignee/checkoutRunId` 조건, `executionRunId` 조건, stale-run 인계(adopt) 경로까지 포함한다. **코드 1**이 그 의사코드를 SQL 한 문장으로 압축한다 — **WHERE** 절의 4개 조건이 “*이 슬롯은 비어 있거나 내가 잡고 있던 것이다*”를 한 트랜잭션 안에서 검증한다.

코드 1. atomic checkout의 의사코드 — 다중 조건 UPDATE 한 번



이로써 **이슈 단일 assignee + atomic checkout** 두 규칙이 합쳐져 “한 이슈 = 한 에이전트가 동시에 잡고 있는 단 한 회차의 run” 이 된다. `checkout_run_id` 와 `execution_run_id` 의 분리는 *오너십 락* 과 *현재 살아 있는 실행 경로* 를 따로 추적하기 위한 것이다.

6. 80여 개 테이블 그룹 지도

전체 스키마는 표 2 처럼 6개 도메인으로 군집된다.

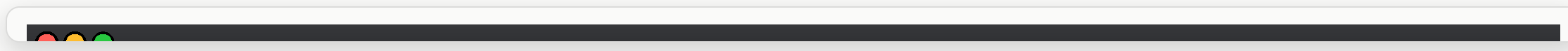
표 2 80여 개 테이블의 도메인 군집

도메인	핵심 테이블	부속 테이블 (예시)
Foundation	<code>companies</code> , <code>users</code> (Better Auth), <code>instance_*</code> , <code>auth.*</code>	<code>company_memberships</code> , <code>instance_user_roles</code> , <code>cli_auth_challenges</code> , <code>invites</code> , <code>join_requests</code> , <code>board_api_keys</code> , <code>cloud_upstreams</code> (#6548 신규 — 마이그레이션 <code>0089</code> , local → cloud 회사 동기화)
Agents & runtime	<code>agents</code> , <code>agent_api_keys</code> , <code>heartbeat_runs</code>	<code>agent_config_revisions</code> , <code>agent_runtime_state</code> , <code>agent_task_sessions</code> , <code>agent_wakeup_requests</code> , <code>environments</code> , <code>environment_leases</code>
Work	<code>issues</code> , <code>projects</code> , <code>goals</code>	<code>issue_comments</code> , <code>issue_documents</code> , <code>document_revisions</code> , <code>issue_attachments</code> , <code>issue_work_products</code> , <code>issue_relations</code> , <code>issue_thread_interactions</code> , <code>issue_tree_holds</code> , <code>issue_labels</code> , <code>labels</code> , <code>issue_read_states</code> , <code>issue_inbox_archives</code> , <code>inbox_dismissals</code> , <code>project_workspaces</code> , <code>project_goals</code> , <code>routines</code>

7. Drizzle 스키마와 마이그레이션 — Drizzle Studio가 보여 주는 모습

Drizzle ORM 의 스키마-퍼스트(schema-first) 워크플로는 다음과 같다. 코드 2 가 Paperclip 에서 스키마 변경의 3단계 안전 게이트를 보여 준다 — 코드(`schema/*.ts`)를 편집하고 SQL 마이그레이션을 자동 생성한 뒤, 명시적으로 적용한다. 운영 적용은 `pnpm db:migrate` 로 명시적이지만, dev/server boot 경로(`server/src/index.ts`)에는 pending migration 자동 적용 조건이 있어 운영 문맥에 따라 동작이 달라진다.

코드 2. Drizzle 스키마-퍼스트 마이그레이션 사이클



생성된 마이그레이션은 `packages/db/src/migrations/` 아래에 누적된다(`packages/db/drizzle.config.ts:5`). 그림 5 는 Drizzle 팀이 공식적으로 제공하는 Drizzle Studio 스크린샷으로, 스키마 시각화와 데이터 편집을 같은 도구에서 할 수 있음을 보여 준다(자세한 내용은 [docs/research/06-drizzle-pglite.md](https://docs.drizzle.team/06-drizzle-pglite.md)).

그림 5. Drizzle Studio (출처: orm.drizzle.team/drizzle-studio/overview)

8. 임베디드 PostgreSQL — 무설정 dev

`DATABASE_URL` 이 비어 있으면 Paperclip 은 자동으로 `~/.paperclip/instances/default/db/` 에 **embedded PostgreSQL** 인스턴스를 띄운다. 이 패키지는 `package.json` 에서 `embedded-postgres@18.1.0-beta.16` 으로 핀 (pin)되어 있고, 한 줄짜리 패치(`patches/embedded-postgres@*.patch`) 가 적용된다. PostgreSQL 17 (full native) 을 그대로 호스트하므로 PGLite (WASM) 보다 호환성이 좋다.

그림 6 은 비교 대상 PGLite (ElectricSQL) 의 데모 스크린샷이다. WASM 으로 브라우저에도 실행 만큼 가벼운 대안이지만, Paperclip 은 데스크톱 dev 의 무게라면 native PostgreSQL 을 선호한다.

그림 6. PGLite — WASM PostgreSQL (출처: github.com/electric-sql/pglite)

운영 환경 전환은 `DATABASE_URL` 환경 변수 하나로 끝난다. **코드 3** 은 자주 쓰는 두 가지 외부 PostgreSQL 연결 — 로컬 Docker, Supabase pooler — 의 URL 형태를 보여 준다. 같은 코드/스키마/마이그레이션이 양쪽 모두에서 그대로 동작한다.

9. 요약

Paperclip의 데이터 모델은 회사 스코프, 단일 assignee, atomic checkout, 비용 귀속, 감사 로그를 중심으로 구성된다.

[03-runtime-execution.md](#)는 이 모델 위에서 heartbeat scheduler · atomic checkout · stranded-work 회복 · silent-run watchdog이 어떻게 동작하는지 분석한다.

Runtime Execution — Heartbeat • Checkout • Watchdog

1. 한 회차의 heartbeat가 만드는 일들

Paperclip의 실행 의미론은 한 가지 단순한 그림으로 압축된다 — **scheduler/wake claim → server-side atomic checkout → adapter.execute → agent runs → cost report → live event push** 의 직선이다. 그림 3-1이 6개 라이프라인(Scheduler · Adapter · Agent runtime · Paperclip API · PostgreSQL · WebSocket→UI) 위에 그 흐름을 시간 축으로 펼쳐 보여 준다.

그림 3-1. heartbeat 한 회차의 시퀀스 — 6 라이프라인을 가로지르는 $t_0 \sim t_6$ 흐름

t_0 에 wake 큐가 만기 도래한 에이전트의 run을 클레임한다. t_1 에 Paperclip server-side에서 atomic checkout이 이슈를 잡는다(`server/src/services/issues.ts:4659-4809`). t_2 에 서버가 `adapter.execute(ctx)` 를 호출해 외부 에이전트(Claude Code, Codex, OpenCode 등)를 spawn한다. t_3 부터 에이전트는 thin ping으로 받은 컨텍스트를 가지고 일을 한다. t_4 에 토큰 사용량이 cost event로 들어와 회사·에이전트·이슈 차원으로 롤업되고, 이 변화는 회사 단위 WebSocket 라이브 채널(`activity.logged · heartbeat.run.*`)로 보드 UI에 즉시 반영된다. t_6 에 결과가

AI Odyssey `in_review` 로 핸드오프되고 실행 락(`execution_run_id`)이 해제된다 — 오너십 락(`checkout_run_id`)은 별도 회복 30 / 98

그치에 따라 유지, 해제된다. 만약 어디선가 예상 하드를 넘으면 t_{60} 의 빨간 점선처럼 모든 흐름이 각목채진다. 그림 3-1의

2. Heartbeat의 정의 — 일반 분산 패턴 한 컷

이 그림이 새로 발명한 것은 거의 없다. 분산 시스템에서 heartbeat 는 오랜 패턴이다. **그림 3-1-A** 는 Martin Fowler 의 *Patterns of Distributed Systems* 시리즈가 정리한 HeartBeat 패턴 — 서버가 주기적인 신호로 자신의 생존을 외부에 알리는 모델 — 의 공식 다이어그램이다.

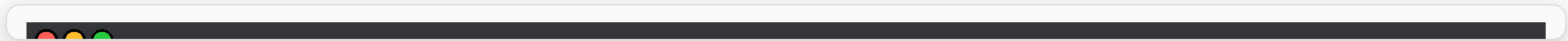
그림 3-1-A. HeartBeat 패턴 (출처: martinfowler.com/articles/patterns-of-distributed-systems/heartbeat.html)

Paperclip 이 비튼 점은 두 가지다. (1) 에이전트가 보내는 heartbeat 가 아니라 **Paperclip 이 에이전트에게 보내는 wake 신호** 다. (2) 한 회차의 wake 가 곧 한 회차의 작업 사이클이고, 그 사이클의 출발 시점에 atomic checkout 이 강제된다. 자세한 패턴 비교는 docs/research/09-sse-heartbeat-patterns.md 에서 다룬다.

3. 어댑터 계약 — 장기 SPEC과 현재 구현 두 층

어댑터 계약은 두 층으로 봐야 한다. 장기 SPEC(`doc/SPEC.md:207-215`)은 3-메서드(`invoke / status / cancel`)를 최소 인터페이스로 그리지만, 현재 구현의 `ServerAdapterManager` (`packages/adapter-utils/src/types.ts:349-352`)에서 필수 메서드는 두 개 — `execute(ctx)` 와 `testEnvironment(ctx)` — 다. 나머지는 옵션이다. 코드 1이 두 층을 한눈에 보여 준다.

코드 1. 어댑터 계약 — SPEC vs 현재 구현



런타임은 `adapter.execute(ctx)` 로 들어간다(`server/src/services/heartbeat.ts` 의 invocation 경로). **이 작은 표면이 multi-runtime 지원의 비밀** 이다 — Python 스크립트, Claude Code, Codex CLI, OpenClaw 게이트웨이 모두 두 필수 메서드만 구현하면 된다. 비용 보고나 작업 상태 갱신은 어댑터 인터페이스에 들어 있지 않다 — 그것은 **에이전트가 Paperclip REST API 를 호출** 해서 한다. 그 API 콜은 어댑터 종류와 무관하다. 즉 어댑터는 **어떻게 실행을 시작·검증하느냐** 만 정의하고, 도메인 작업은 통일된 API 표면에서 일어난다.

4. 7-state issue state machine

이슈는 7개 상태를 가진다. 그림 3-2 가 그 전체 전이망과 회복 규칙을 한 페이지로 보여 준다.

그림 3-2. 이슈 7-state state machine — 정상 전이와 회복 경로

전이의 핵심은 다음과 같다.

- `backlog` → `todo`: 보드/CEO 가 일을 살리는 행위
- `todo` → `in_progress`: 반드시 **atomic checkout** 을 거쳐야 한다
- `in_progress` → `in_review`: 결과 핸드오프
- `todo` → `blocked` 또는 `in_progress` → `blocked`: blocker 선언, 또는 stranded recovery / watchdog 격상이 실행·대기 실패를 `blocked` 로 표면화한다(`doc/SPEC-implementation.md:421-426`).
- `blocked` → `in_progress`: 마지막 blocker가 `done` 이 되면 `issue_blockers_resolved` wake가 큐잉되고, 그 wake가 fire한 run의 checkout이 성공할 때 비로소 전환된다 — DB 상태가 즉시/자동 전이되는 것이 아니다.

5. checkout vs execution — 두 ID의 분리

`issues.checkout_run_id` 와 `issues.execution_run_id` 가 분리된 이유는 다음 표가 가장 명료하다.

표 1 두 ID 의 분리 이유

컬럼	의미	예시
<code>checkout_run_id</code>	오너십 락 — 누가 지금 이 이슈를 잡고 있는가	에이전트 X 가 잡았고, 다음 wake 도 X 에게 가야 함
<code>execution_run_id</code>	현재 라이브 실행 — 지금 살아 있는 run 의 ID	X 의 이전 run 은 죽었지만 락은 여전히 X. 새로 wake 가 들어오면 새로운 <code>execution_run_id</code> 를 채운다

표 1 의 두 컬럼이 둘 다 있어야 보수적 회복이 작동한다 — `checkout_run_id` 만 있으면 죽은 run 도 영원히 락을 잡고 있는 셈이고, `execution_run_id` 만 있으면 누구의 작업이었는지의 기억이 사라진다. 이 분리 덕분에 startup recovery 시 “오너는 X 가 맞지만, 살아 있는 실행은 사라졌다. 같은 X 에게 한 번 더 wake 를 보내자” 같은 정확한 회복이 가능하다. 그림 3-2 에서 `in_progress` 노드가 자기 자신으로 돌아오는 작은 화살표는 이 같은 X 에게 한 번 더의 시각적 표현이다.

6. Stranded recovery — 보수적 한 번 재시도

`doc/execution-semantics.md` § 8 은 두 가지 stranded 패턴을 정의한다.

- **Stranded `todo` (8.1):** `todo` 상태에서 dispatch wake 가 죽고 큐에 남은 wake 도 없는 경우.
- **Stranded `in_progress` (8.2):** `in_progress` 상태에서 라이브 run 이 사라지고 큐에 continuation 도 없는 경우.

회복 규칙은 동일하게 보수적이며 2단계다. **1단계:** 자동으로 한 번만 wake 를 재큐잉한다. **2단계:** 그 회차가 또 stranded 로 끝나면 이슈를 `blocked` 로 옮기고, source-scoped `issue_recovery_actions` 행을 upsert한다

(`packages/db/src/schema/issue_recovery_actions.ts:16-67` , `server/src/services/issue-recovery-actions.ts:1-295`). 활성(`active` · `escalated`) recovery action은 source issue당 하나로 제한되고

(`packages/db/src/schema/issue_recovery_actions.ts:61-63`), 같은 source issue 안에서는 (`cause`, `fingerprint`) 조합으로 중복 upsert를 막는다(`:64-66`). `owner_*` / `wake_policy` / `monitor_policy` 컬럼으로

누가 다음에 무엇을 해야 하는가 를 명시한다. 절대 다른 에이전트로 재할당하지 않는다. 이는 § 12 가 명시한 의도다 —

“preserve ownership / retry once / create explicit recovery action / escalate visibly.”

7. Silent active-run watchdog

라이브 process 가 살아 있어 보이지만 출력이 멈춘 경우는 별도의 회로가 잡는다. recovery 서비스가 다음 5개 분류로 silence 를 라벨링한다 — `ok`, `suspicious`, `critical`, `snoozed`, `not_applicable`. 신호는 *recovery action* 으로 표면화되며, issue-backed 평가가 필요한 경우 `stale_active_run_evaluation` 이슈를 만든다. `suspicious` 는 medium-priority evaluation으로, `critical` 은 high-priority 로 격상되면서 원본 이슈를 평가 작업에 block 한다. 이 흐름의 의도된 결과는 — 프로세스를 함부로 죽이지 않으면서, 보드와 회복 오너에게 *evaluation* 으로 드러내는 것 — 다음 두 줄에 압축된다.

“Suspicious silence creates a medium-priority review issue. Critical silence raises that review issue to high priority and blocks the source issue on the explicit evaluation task without cancelling the active process.” —

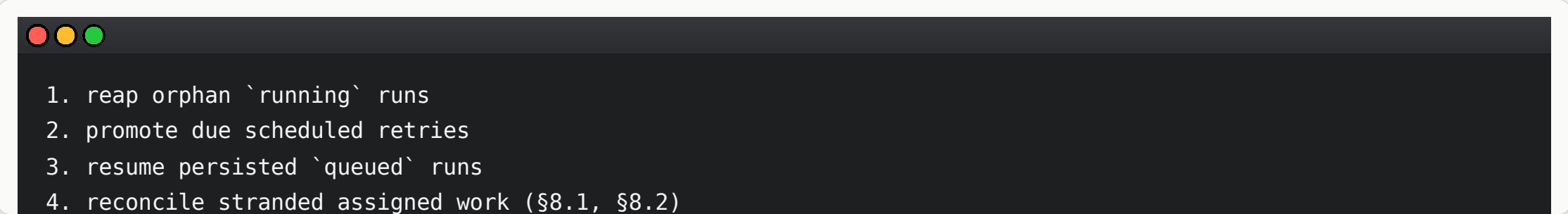
`doc/execution-semantics.md` § 10

`continue` · `snooze` · `dismissed_false_positive` 세 가지 결정만 합법이며, 결정 권한은 보드 또는 평가 이슈의 assignee 에게만 있다.

8. Startup reconciliation — 7단계

API 서버가 부팅되면 startup/periodic reconciliation 이 결정론적 순서로 돈다(`server/src/index.ts:676-780`). 코드 2 가 그 순서다 — stranded assigned work reconciliation 과 silent active-run watchdog 은 서로 다른 실패 모드(공식 dispatch가 죽은 경우 vs 라이브 run이 살아 보이지만 출력이 멈춘 경우)를 *순서대로* 다룬다.

코드 2. 부팅 시 startup reconciliation 순서

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains a numbered list of four steps for startup reconciliation.

```
1. reap orphan `running` runs
2. promote due scheduled retries
3. resume persisted `queued` runs
4. reconcile stranded assigned work (§8.1, §8.2)
```

이 결정론적 순서가 “재부팅이 운영자의 인지를 흐뜨리지 않는다” 는 약속의 기반이다.

9. heartbeat가 아닌 것들

마지막으로, 안티 정의가 중요하다.

- 고정 빈도 cron 이 아니다: 실제로는 큐에 쌓인 wake 와 monitor next-check-at 으로 트리거된다.
- 어댑터 상태 폴링이 아니다: 현재 구현은 별도 `status()` 메서드 대신 `heartbeat_runs.status`, process metadata, output event, `adapter.execute` 결과, watchdog evidence를 조합해 liveness를 판단한다 (`packages/db/src/schema/heartbeat_runs.ts:14-40`). 출력 silence 가 진짜 평가 신호다.
- 자동 재할당이 아니다: 누가 잡았는가 는 회복을 거치며 보존된다.
- 자동 완료가 아니다: prose 코멘트로 done 을 추론하지 않는다.

이 보수적 입장이 § 12의 결론이다. [04-adapters-and-skills.md](#)는 이 실행 골격을 채우는 built-in 어댑터와 skills 시스템을 분석한다.

Adapters & Skills — built- in 어댑터 + Paperclip Skill

1. “호출 가능한 것은 모두 채용된다”

Paperclip README의 슬로건 — “*If it can receive a heartbeat, it’s hired.*” — 은 어댑터 시스템의 핵심 약속이다. 어댑터는 Paperclip control plane과 외부 에이전트 런타임 사이의 좁은 통로다.

그림 7 이 런타임 전용 built-in 어댑터 11개와 범용 어댑터 2개(`process` · `http`)를 능력 축으로 정렬한다 (`server/src/adapters/registry.ts:511-524`). 각 축은 `ServerAdapterModule` (`packages/adapter-utils/src/types.ts:349-431`)의 실제 필드 — 핵심 실행·모델·옵션 필드(`execute` , `testEnvironment` , `listSkills` / `syncSkills` , `sessionCodec` , `supportsLocalAgentJwt` , `getQuotaWindows` , `detectModel` , `getConfigSchema` 등은 `:349-396`)와 capability flags(`supportsInstructionsBundle` , `instructionsPathKey` , `requiresMaterializedRuntimeSkills` , `getRuntimeCommandSpec` 등은 `:407-431`) — 에 대응한다.

그림 7. 어댑터 능력 매트릭스 — 런타임 전용 built-in 11개 + 범용 process/http × `ServerAdapterModule` 필드

전반적으로 주요 local coding adapters(`claude-local` , `codex-local` , `cursor-local` , `gemini-local` , `opencode-local` , `pi-local`)는 `execute` + `testEnvironment` 외에 `listSkills` / `syncSkills` , `sessionCodec` ,

2. built-in 어댑터 요약

각 어댑터의 호출 시그니처와 라이선스/과금 성격은 다음과 같다(자세한 내용 + 인용은 [docs/research/04-headless-coding-agents.md](#)).

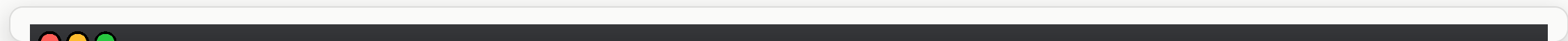
표 1 런타임 전용 built-in 10종 + 범용 2종

어댑터	대상 런타임	호출 형태	MCP	라이선스 / 요금
<code>claude-local</code>	Anthropic Claude Code	<code>claude -p --output-format stream-json</code>	●	proprietary CLI · API or Claude Pro/Max
<code>codex-local</code>	OpenAI Codex CLI	<code>codex exec --json -- model ...</code>	●	Apache-2.0 · ChatGPT plan or API
<code>cursor-local</code>	Cursor CLI / agent	<code>cursor-agent ...</code>	●	closed-source · BYOK
<code>cursor-cloud</code>	Cursor Cloud Agent (Cursor SDK / Cloud Agents API v1)	managed remote agent	◐	proprietary · BYOK
<code>gemini-local</code>	Google Gemini CLI	<code>gemini --output-format</code>	●	Apache-2.0 · free Google login or

3. 어댑터 폴더의 4개 진입점

`packages/adapters/<name>/package.json` 의 `exports` 필드는 모든 어댑터가 동일한 4개 진입점을 노출함을 강제한다. **코드 1** 이 그 표준 export 맵이다 — 같은 npm 패키지가 `server·ui·cli` 에 *각자 다른 모듈* 을 제공하는 구조이며, 이렇게 해야 server 빌드는 React 폼을, UI 빌드는 spawn 로직을 *불러오지 않게* 된다.

코드 1. 어댑터 패키지의 표준 `exports` 맵 (4개 진입점)



이 분할은 코드 위생과 번들 크기 모두에 좋다. 서버는 spawn 로직만, UI 는 React 폼만, CLI 는 onboarding helper 만 가져간다. **표 2** 가 각 진입점의 책임을 정리한다.

표 2 어댑터 4개 진입점의 책임

진입점	임포트하는 쪽	무엇을 export 하나
· AI Odyssey	server / ui / cli 모두	<code>id</code> , <code>displayName</code> , <code>defaultRuntimeConfig</code> , <code>configSchema</code> , type-safe 메타

4. 외부 어댑터 플러그인 — 코어 미수정 확장

리포지터리 상단의 `adapter-plugin.md` 가 묘사하는 외부 어댑터 플러그인 흐름은 다음과 같다. **코드 2** 가 외부 어댑터를 코어 수정 없이 *설치* → *등록* → *활성화* → *사용* 4단계로 들이는 표준 절차다 — 1단계는 CLI 한 줄, 2\~4단계는 보드 UI 안에서 마우스 클릭으로 끝난다.

코드 2. 외부 어댑터 플러그인 lifecycle — 4단계



외부화 상태. 현재 upstream `master` 의 `server/src/adapters/registry.ts:128\~138` 은 `hermes-paperclip-adapter` 를 정적 **import**하고, 같은 파일의 built-in 등록 배열에 `hermesLocalAdapter` 를 포함한다. 즉 메인 라인은 아직 Hermes를 코어 의존성으로 포함한다. 반면 HenkDz 포크의 `feat/externalize-hermes-adapter` 브랜치는

“코어에는 `hermes` 의존성도 등록도 두지 않고, 플러그인으로만 도입”

하는 외부화 사례다. 이 차이를 분리해서 읽어야 adapter plugin 설계가 어디까지 구현됐고, 어디부터 브랜치별 정책인지 혼동하지 않는다.

5. Paperclip Skill — 에이전트가 Paperclip을 다루는 방법

어댑터 옆에 Paperclip 이 *에이전트* 안쪽으로 보내는 통로가 있다 — `skills/paperclip/` 디렉터리에 들어 있는 **Paperclip SKILL.md**. SKILL.md 는 Claude Code · Codex 등이 채택한 skill 표준 포맷의 마크다운 파일이며, 에이전트에게 다음을 가르친다.

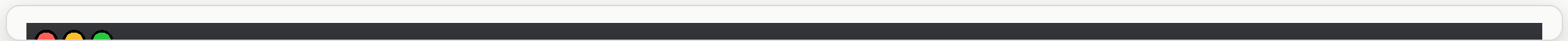
- 작업(issue) CRUD — `paperclipai issues list / view / update`
- 진척 보고 — `paperclipai issues update <id> --status in_progress`
- 회사 컨텍스트 읽기 — 골 트리, org chart, 현재 상태
- 비용 보고 — 토큰 / API 사용량 기록
- 인터-에이전트 통신 규칙 — cross-team 작업 수락·거절 프로토콜

`skills/` 폴더에는 7개 추가 스킬이 같이 산다. 코드 3 이 그 디렉터리 트리이며, 각 스킬은 *어떤 상황에서* 에이전트가 어떻게 행동해야 하는지를 마크다운으로 가르친다.

6. MCP 서버 — 또 다른 통로

위의 SKILL.md 는 읽는 통로이고, `packages/mcp-server` 는 호출하는 통로다. Paperclip 은 자체 MCP 서버를 묶어 배포해, MCP 호환 호스트(Claude Code · Codex · Cursor · Claude Desktop)가 도구 호출 한 번으로 Paperclip API 를 다룰 수 있게 한다. 코드 4 가 그 호출 경로를 한 줄로 압축한다 — agent 가 자기 어댑터 종류와 무관하게 같은 MCP 도구를 부르고, 그 도구는 MCP 서버를 통해 Paperclip REST API 로 풀린다. 자세한 MCP 모델은 [docs/research/02-model-context-protocol.md](https://github.com/ai-odyssey/paperclip/blob/main/docs/research/02-model-context-protocol.md) 에서 다룬다.

코드 4. MCP 호출 경로 — agent → MCP server → REST → DB



이 두 층(SKILL.md + MCP)은 현재 어댑터 실행 계약(`execute` / `testEnvironment`)과 **수직 직교** 한다. 어댑터는 *프로세스 차원*, SKILL/MCP 는 *도메인 차원* 이다.

[05-server-api.md](https://github.com/ai-odyssey/paperclip/blob/main/docs/server-api.md)는 이 모든 진입점을 한데 묶는 Express 본체 — routes / services / realtime / auth — 를 분석한다.

Server & API — Express · Routes · Services · WebSocket

1. 한 호스트, 두 종류의 클라이언트

Paperclip API의 첫 번째 결정은 단순하다 — **하나의 REST 표면, 두 가지 인증**이다. 같은 `/api/...` 엔드포인트가 (1) 보드 운영자(웹 UI · CLI)와 (2) 에이전트(API key)를 동시에 받는다. 어떤 키로 들어왔는가가 권한을 결정한다. 이는 SPEC § 8의 “*Single unified REST API*” 결정이다.

그림 3이 `server/src`의 내부 토폴로지를 한 페이지로 그려 준다.

그림 3. `server/src`의 내부 토폴로지 — `auth · middleware · realtime · routes · services · adapters · DB`

`inbound ( app.ts · auth/ · middleware/ · realtime/ )`에서 들어와 → `routes/`로 분기 → `services/`가 도메인 로직을 담당 → `adapters/`가 외부 런타임을 부르고 → `DB · storage · secrets`가 영속화한다. 모든 mutating 작업은 `activity_log`에 한 줄을 남긴다. 그림 3의 핵심 관찰점은 두 가지다. (1) `routes/`는 “얇은 컨트롤러”로, `services/`는 “두꺼운 도메인”으로 그려져 있다. 이는 § 3의 디렉터리 트리에서도 확인되는 비대칭이며, 라우트 한 줄을 바꾸는 일보다 도메인 로직 변경에 4개 레이어(`db/shared/server/ui`) 동기화가 더 자주 따라오는 이유다. (2) `realtime/`은 같은 HTTP 서버를 공유하지만 *별도 inbound 경로*다. WebSocket 라이브 채널은 Express 라우트가 아니라 HTTP server의 `upgrade`

2. routes/ — 엔드포인트 카탈로그

`server/src/routes/`에는 36개의 라우트 파일이 있다. 표 1 이 보드 운영자 시점에서 의미 있는 것을 묶는다.

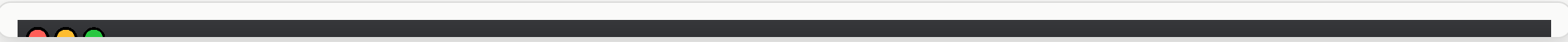
표 1 주요 REST 엔드포인트 군

라우트 그룹	책임	대표 엔드포인트
<code>companies.ts</code>	회사 lifecycle	<code>GET/POST /api/companies</code> , <code>PATCH /api/companies/:id</code>
<code>agents.ts</code> (120 KB)	에이전트 CRUD · 상태	<code>POST /api/companies/:companyId/agents</code> , <code>POST /api/agents/:id/pause</code>
<code>issues.ts</code> (205 KB)	이슈 풀 lifecycle + recovery action	<code>GET/POST /api/companies/:companyId/issues</code> , <code>PATCH /api/issues/:id</code> , <code>POST /api/issues/:id/checkout</code> , <code>POST /api/issues/:id/release</code> , <code>POST /api/issues/:id/comments</code> , <code>GET /api/issues/:id/recovery-actions</code> , <code>POST /api/issues/:id/recovery-actions/resolve</code>
<code>issue-tree-control.ts</code>	tree hold · 통제	<code>POST /api/issues/:id/tree-holds</code> , <code>GET /api/issues/:id/tree-control/state</code> (<i>blocker</i> 추가/제거는 별도 라우트가 아니라 <code>PATCH /api/issues/:id</code> 의 <code>blockedByIssueIds</code> payload)

3. services/ — 도메인 오케스트레이션

라우트가 *얕은 컨트롤러* 라면, services/ 는 *두꺼운 도메인* 이다. 디렉터리 구성을 5개 도메인으로 묶으면 다음과 같다. 코드 1 이 그 5개 도메인 트리이며, 각 군집의 KB 표기가 어디에 도메인 무게가 쏠려 있는지를 그대로 보여 준다 — routes/issues.ts 205 KB(§ 2 표 1)· company-portability.ts 178 KB 가 가장 큰 두 봉우리.

코드 1. server/src/services/ 의 5-도메인 디렉터리



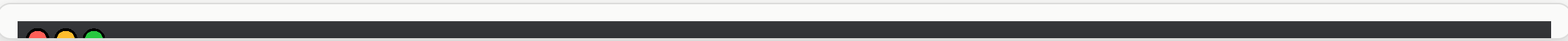
company-portability.ts 가 178 KB 인 점이 흥미롭다 — 회사 전체를 *템플릿* 또는 *스냅샷* 으로 export/import 하는 로직이 거기 모여 있다. 이는 SPEC § 2 의 “*Exportable Org Configs*” 약속의 구현부다.

신규 도메인 — Cloud Upstream sync(#6548, 2026-05-22) — services/cloud-upstreams.ts 가 +1298 LOC 로 모놀리식 추가되며 한 도메인 군집이 통째로 들어왔다. 이는 로컬 Paperclip 인스턴스를 **cloud upstream**(paperclip.ing 또는 셀프 호스트 cloud) 과 연결해 회사·이슈·run 산출물을 push 동기화하는 통로다. 데이터 모델 차원에서 새 테이블 cloud_upstreams 가 추가됐고(마이그레이션 0089_cloud_upstreams , 스키마 packages/db/src/schema/cloud_upstreams.ts:1-75), shared 타입 packages/shared/src/types/cloud-

4. realtime/ — WebSocket 라이브 이벤트 채널

UI 가 보드를 보고 있는 동안 새 cost event 가 들어오거나 이슈 상태가 바뀌면, 사용자는 새로 고침 없이 그 변화를 봐야 한다. Paperclip 은 이를 **WebSocket** 으로 푼다 (`server/src/realtime/live-events-ws.ts:63-176`) — Express 라우트가 아니라 HTTP server의 `upgrade` handler에 붙는 `ws` 서버다. UI 는 `LiveUpdatesProvider` 가 회사 단위로 한 소켓을 열어 두고, 들어온 이벤트로 TanStack Query 캐시를 무효화한다. 코드 2 가 두 종류의 클라이언트가 같은 WebSocket 엔드포인트를 어떻게 다른 인증 방식으로 부르는지 보여 준다 — 브라우저는 *세션/local_trusted*, 헤드리스는 *agent API key Bearer*.

코드 2. WebSocket 라이브 채널 핸드셰이크 — 브라우저 vs 헤드리스



브라우저는 [WebSocket API](#) 가 사용자 정의 헤더 첨부을 허용하지 않으므로 *세션/로컬 신뢰* 경로 만 가능하고, 헤드리스 클라이언트는 *agent API key Bearer* 로 인증한다. board API key는 REST의 `actorMiddleware` 에서만 허용되고 WebSocket upgrade 경로의 Bearer는 현재 agent API key 전용이다. 서버는 두 경로 모두를 같은 회사 스코프로 검증한다.

5. auth/ — Better Auth + board/agent credential 표면

server/src/auth/ 는 [Better Auth](#) 위에 board actor 모드 2종 + agent credential 경로 2종을 엮는다 (server/src/middleware/auth.ts:24-199). 표 3 이 그 네 표면이다.

표 3 인증 주체와 credential 경로

주체	어떻게 들어오는가	어디까지 볼 수 있는가
보드 사용자 (사람)	Better Auth 세션 쿠키 (<code>authenticated</code> 모드)	자기 회사들의 모든 데이터, instance admin 이라면 인스턴스 전역
<code>local-board</code> 가짜 사용자	<code>local_trusted</code> 모드 — 로그인 없음	단일 운영자 시나리오 (로컬 데스크톱)
보드 API key	<code>board_api_keys</code> (해시 저장) → <code>Authorization: Bearer ...</code>	CLI · CI · 자동화용. 보드 사용자 권한 범위
에이전트	<code>agent api keys</code> → <code>Authorization: Bearer agent ...</code>	자기 회사 내 자기 권한

6. middleware/ — 회사 경계와 redaction

라우트 본체에 도달하기 전 미들웨어가 다음을 강제한다.

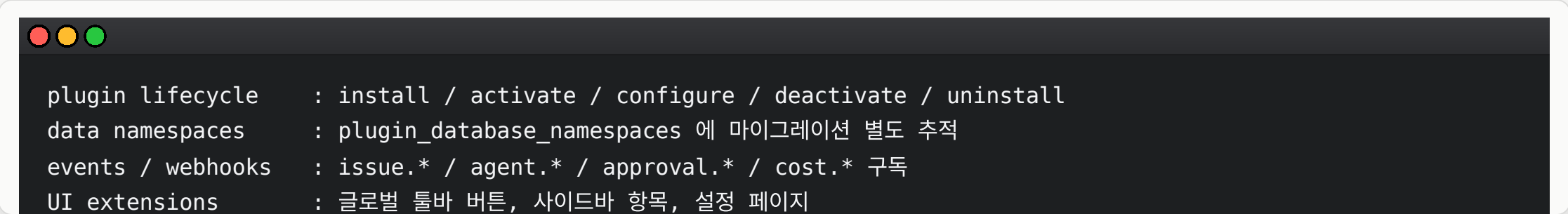
1. **회사 멤버십** — URL 의 `:companyId` 가 현재 주체에 보이는지 검사
2. **에이전트 키 격리** — `agent_api_keys` 는 자기 회사만, 자기 에이전트 본인의 권한 범위만
3. **로그 redaction** — 비밀 값(secrets, env)이 활동 로그에 새지 않도록 마스킹
4. **에러 매핑** — 4xx / 409(conflict) / 422(validation) / 500 을 일관되게 변환

이 미들웨어 체인은 `AGENTS.md` 의 § 8 *API and Auth Expectations* 섹션이 강제하는 약속과 1:1 대응한다.

7. 설치형 플러그인 런타임

`packages/plugins`가 정의한 plugin SDK 위에 `server/src`가 다음 hook을 구현한다. 코드 3 이 그 6가지 hook 카테고리다 — 언제 들어오고, 어디에 데이터를 두며, 무엇을 들을 수 있고, 어떤 UI 영역을 채우며, 새 어댑터까지 등록하고, 회사 자원을 *manifest*로 선언해 동기화할 수 있는지 한 페이지에 정리.

코드 3. plugin SDK의 6가지 hook 카테고리

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It displays a list of plugin SDK hooks and their descriptions.

```
plugin lifecycle      : install / activate / configure / deactivate / uninstall
data namespaces       : plugin_database_namespaces 에 마이그레이션 별도 추적
events / webhooks     : issue.* / agent.* / approval.* / cost.* 구독
UI extensions         : 글로벌 톨바 버튼, 사이드바 항목, 설정 페이지
```

이 통로 덕분에 *지식 베이스 · 매출 회계 · 새 어댑터* 같은 V1 범위 밖 기능을 코어 수정 없이 도입할 수 있다.

8. 요약

Paperclip의 서버는 “한 표면, 두 사용자, 5개 서비스 도메인”이다. Express 라우트는 얇고, services/가 두껍고, 라이브 이벤트는 회사 단위 WebSocket 한 줄로 흐르며, 플러그인 런타임이 코어 수정 없는 확장을 허용한다.

[06-ui-and-board.md](#)는 보드 UI — React + Vite + shadcn/ui — 가 이 표면을 어떻게 그리는지 분석한다.

UI & Board — React · Vite · shadcn/ui

1. UI의 위치

`ui/` 는 보드 운영자가 일상적으로 머무는 single page application 이다. 기술 스택은 다음과 같다
(`ui/package.json:30-78`) — **Vite 6 + React 19 + TypeScript + Tailwind CSS 4 + shadcn/ui + Radix UI + TanStack Query 5 + react-router-dom 7**. 같은 프로젝트에 Storybook 이 별도 디렉터리로 동거(`ui/storybook/`) 한다. 프론트엔드 기술 토대의 자세한 설명은 [docs/research/08-react-vite-shadcn.md](https://docs.research/08-react-vite-shadcn.md).

shadcn/ui 의 공식 **dashboard-01** 블록 은 이 톤의 표준 모범이다 — Paperclip 보드 UI 는 그 위에 회사·이슈·코스ٹ 도메인을 입혔다. 그림 6-1 은 *외부 인용 다이어그램* 으로, Paperclip 자체 스크린샷이 아니라 디자인 톤의 *원형* 을 보여 준다 — 좌측 사이드바 + 상단 헤더 + 그리드 카드 위주의 대시보드 레이아웃이 Paperclip `/dashboard` 와 1:1 로 이어진다.

그림 6-1. shadcn/ui 공식 dashboard-01 블록 (출처: ui.shadcn.com)

2. 70+ 페이지의 라우팅 표

`ui/src/App.tsx` 에 등록된 페이지를 도메인으로 묶으면 다음과 같다(편의상 일부만 발췌).

표 1 주요 페이지 라우팅

도메인	경로	페이지 컴포넌트	책임
대시보드	<code>/dashboard</code> , <code>/dashboard/live</code>	<code>Dashboard.tsx</code> , <code>DashboardLive.tsx</code>	한눈 피드
회사	<code>/companies</code> , <code>/company/settings</code> , <code>/company/settings/{environments,access,invites,secrets}</code>	<code>Companies.tsx</code> , <code>CompanySettings.tsx</code> , <code>CompanyAccess.tsx</code> , <code>CompanyEnvironments.tsx</code> , <code>CompanyInvites.tsx</code> , <code>Secrets.tsx</code> (2,155 LOC)	회사 secret
회사 export/import	<code>/company/export/*</code> , <code>/company/import</code>	<code>CompanyExport.tsx</code> , <code>CompanyImport.tsx</code>	템플

3. 디렉터리 구조

코드 1 이 `ui/src/` 의 표준 트리다 — 두 확장점은 로딩 계약이 다르다. `adapters/` 는 built-in 어댑터 UI를 정적 registry에 등록(`ui/src/adapters/registry.ts`)하고, 외부 어댑터의 동적 부분은 `/api/adapters/:type/ui-parser.js` 를 fetch해 sandboxed Web Worker 안에서 parser source를 실행하는 *parser bridge* 방식이다 (`ui/src/adapters/dynamic-loader.ts`). 반면 `plugins/` 는 `ui/src/plugins/slots.tsx` 에서 dynamic ESM `import()` 로 플러그인 UI를 로딩한다. `api/` 가 `packages/shared` 의 zod 스키마를 그대로 import 해 *타입 끝까지 안전* 의 모노레포 이득을 챙기는 점은 두 확장점이 공통이다.

코드 1. `ui/src/` 디렉터리 구조

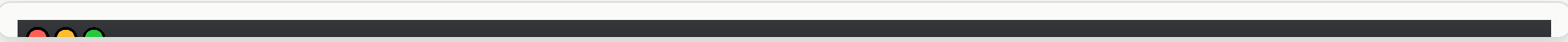


```
api/ 가 packages/shared 의 zod 검증을 그대로 import 해 type 안전성을 끝까지 유지한다는 점이 모노레포 구성의 큰 이득이다. server / shared / ui 가 같은 타입 정의를 공유하므로, 스키마 변경이 컴파일 시점에 모든 클라이언트에 전파된다.
```

4. 데이터 플로우 — TanStack Query + WebSocket

서버 상태는 **TanStack Query** 가 캐시·재시도·자동 invalidation 을 담당한다. `LiveUpdatesProvider` 가 회사 단위 WebSocket 한 소켓을 열어 두고, 들어온 `LiveEvent` 의 `type / entity_type` 으로 query key 를 invalidate 해 즉시 새 fetch 가 흐르는 구조다. **코드 2** 가 그 패턴을 `useQuery + useEffect(Websocket)` 한 쌍으로 압축한다 — 캐시는 TanStack Query 가, push invalidation 은 라이브 채널이.

코드 2. TanStack Query + WebSocket 라이브 invalidation 패턴



이 구조 덕분에 *클라이언트 캐시 + push invalidation* 의 두 축이 합쳐져, 보드 UI 는 폴링 없이 즉시 반응한다.

다만 이 방식은 이벤트 자체가 최종 데이터를 담는 event sourcing 모델이 아니다. WebSocket 메시지는 대부분 “무엇이 바뀌었는가”를 알려 주고, 실제 화면 데이터는 TanStack Query가 다시 REST API로 가져온다. 장점은 클라이언트 상태 복제가 단순하다는 점이고, 한계는 네트워크가 불안정할 때 짧은 순간 stale view가 보일 수 있다는 점이다. 그래서 UI가 mutation 실패를 조용히 삼키지 않고, invalidate 이후 refetch 오류를 화면에 드러내는 것이 중요하다. 이는 `AGENTS.md` 의 “Surface failures clearly” 규칙과도 맞물린다.

5. shadcn/ui — “패키지가 아니라 코드 복사”

shadcn/ui 는 **npm 패키지**가 아니다 — `npx shadcn add button` 같은 CLI 가 컴포넌트 소스 코드를 프로젝트에 복사해 넣는 분배 모델이다. Radix UI 의 비스타일 primitives 위에 Tailwind 스타일을 입혀 둔 형태이며, 사용자는 그 코드를 그대로 갖고 자유롭게 수정한다. Paperclip 의 `ui/components/` 는 이 모델을 따른다.

장점은 두 가지다. (1) 의존성 잠금에서 자유롭다 — 라이브러리 마이그레이션이 없다. (2) 디자인 시스템을 회사 색·브랜드에 맞춰 변형할 수 있다. Paperclip 보드 UI 는 회사별 `brand_color` 컬럼(`packages/db/src/schema/companies.ts:28`) 을 `CompanyPatternIcon` 의 패턴 색상과 회사 선택/생성 UI의 색상 스와치에 연결한다.

6. 단일 origin dev 모드 — Express 미들웨어

`server/src/app.ts` 는 dev 환경에서 Vite 를 **middleware** 로 마운트한다. **코드 3** 이 그 부팅 의사코드다 — Express 한 인스턴스가 `/api` 와 SPA 자산을 같은 origin 에서 동시에 서빙한다.

코드 3. dev 모드 — Express 안에 Vite middleware 마운트

```
// 개발 서버 부팅 (의사코드)
const app = express();
```

이 구성의 효과는 다음과 같다. (1) **CORS 가 필요 없다** — UI 의 `fetch('/api/...')` 가 같은 origin 으로 떨어진다. (2) **인증 쿠키도 단순** — Better Auth 세션이 같은 origin 에서 set/read 된다. (3) **빌드 후에는** Express 가 `ui/dist/*` 를 정적으로 서빙한다. (4) **HMR 도 동일 origin** — Vite WS 가 그대로 동작한다.

7. 디자인 가이드 — `/design-guide`

`ui/src/pages/DesignGuide.tsx` (56 KB)는 Paperclip 보드 UI의 디자인 시스템을 한 페이지에 모아 둔 라이브 문서다. 컬러 토큰 · 타이포그래피 · 스테이터스 칩 · 우선순위 시그널 · 컴포넌트 컴포지션 패턴이 함께 정리되어 있다. 신규 컴포넌트는 이 페이지의 패턴을 기준으로 검토된다.

8. Storybook

`ui/storybook/` 가 컴포넌트 단위 카탈로그를 제공한다. `pnpm storybook` 으로 6006 포트에 띄우거나, `pnpm build-storybook` 으로 정적 사이트를 만든다. 컴포넌트 PR 의 시각적 회귀를 잡는 일차 도구다.

9. UI 없이 운영할 수 있는가

실제로는 가능하다. CLI (`paperclipai`) + 에이전트 API + MCP 서버만으로 회사를 만들고 이슈를 만들고 비용을 보고할 수 있다. 그러나 **승인 큐**와 **cost 차트**를 사람의 눈으로 봐야 하는 상황에서는 보드 UI가 사실상 의무다 — 그래서 SPEC § 10의 V1 must-have 안에 Web UI가 명시되어 있다.

[07-governance-cost.md](#)는 Board · Approvals · Budgets · Activity Log의 거버넌스 축을 한 묶음으로 분석한다.

Governance & Cost — Board · Approvals · Budget · Activity Log

1. 사람이 떠나지 않는 자리

자율 에이전트 회사의 핵심 이슈는 *얼마나 자율인가*가 아니라 *어디서 인간이 결정해야 하는가*다. Paperclip은 그 자리를 **Board**라는 이름으로 명시한다. SPEC § 1은 다음을 포함한다.

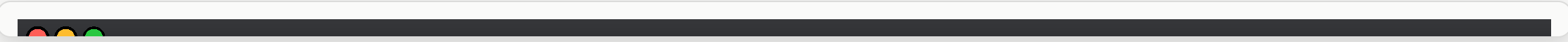
- **승인 게이트** — 신규 에이전트 고용, CEO의 초기 전략 분해
- **항상 가용한 권한** — 회사 예산 설정, 임의의 에이전트/이슈/프로젝트 일시정지·재개, 풀 PM 권한, 모든 에이전트 결정의 override, 임의 단계의 예산 수정

V1의 보드는 *단일* 인간 운영자다(SPEC § 10의 *Single human Board*).

2. 승인 흐름 — `approvals` 테이블

승인은 단일 테이블 `approvals` 로 정규화된다. 실제 컬럼은 다음과 같다(`packages/db/src/schema/approvals.ts`). 코드 1 이 12개 컬럼을 한 화면에 모은 것이며, 대상 식별을 별도 `subject_*` 컬럼이 아니라 `type` + `payload jsonb` 로 표현한 점이 가장 큰 설계 결정이다.

코드 1. `approvals` 테이블의 핵심 컬럼



```
packages/shared/src/constants.ts:373\~388 가 단일 소스다 — APPROVAL_TYPES = [hire_agent,
approve_ceo_strategy, budget_override_required, request_board_approval], APPROVAL_STATUSES =
[pending, revision_requested, approved, rejected, cancelled]. 대상 식별은 별도
subject_type/subject_id 컬럼이 아니라 type + payload jsonb 로 표현되며, 이슈와의 보조 링크는
issue_approvals 테이블이 따로 받는다(run 링크는 이 테이블에 없다). hard budget incident가 만드는
budget_override_required approval처럼 시스템이 만드는 승인은 두 requester가 모두 null이다.
```

3. 거버넌스의 최후 수단 — Pause / Cancel / Override

보드가 평소 보지 않더라도, 다음 hard 권한은 항상 살아 있다.

표 1 항상 가용한 보드 권한

행위	효과	라우트
에이전트 일시정지	<code>agents.status='paused'</code> , 진행 중 run 에 graceful cancel + 미래 heartbeat 봉쇄	POST <code>/api/agents/:id/pause</code>
회사 일시정지	<code>companies.status='paused'</code> — 모든 heartbeat 정지	PATCH <code>/api/companies/:id</code>
이슈/트리 hold	<code>issue_tree_holds</code> 로 부모 트리 잠금	POST <code>/api/issues/:id/tree-holds</code>
직접 수정	임의 이슈/프로젝트의 본문·상태 직접 PATCH	PATCH <code>/api/issues/:id</code>
예산 즉시 변경	회사 / 에이전트 monthly cents	PATCH <code>/api/companies/:companyId/budgets</code> , PATCH <code>/api/agents/:agentId/budgets</code>

4. 예산 — 회사가 통제할 수 있는 가장 큰 변수

LLM 호출 비용은 자율 에이전트 회사의 가장 큰 위험이다. Paperclip은 **3-tier 예산 모델**을 코어로 둔다.

1. **Visibility** — 회사 / 에이전트 / 프로젝트 / 이슈 단위 cost 대시보드.
2. **Soft alert** — 예산의 80% 같은 임계 도달 시 보드에 알림.
3. **Hard ceiling** — 100% 도달 시 자동 일시정지, 보드 알림.

그림 9 가 이 흐름을 6 단계 — Ingestion → Normalization → Attribution → Rollup → Policy → Enforcement — 로 한 페이지에 묶는다.

그림 9. 예산 통제의 6단계 파이프라인 — Ingestion → Normalization → Attribution → Rollup → Policy → Enforcement

그림 9는 왼쪽에서 오른쪽으로 비용이 발생해 정책에 닿는 흐름을 따라 읽으면 된다. 1단계 *Ingestion* 이 어댑터 응답에서 토큰/달러를 거두고, 2단계 *Normalization* 은 adapter/API 가 보고한 `costUsd` 또는 `costCents` 를 ledger의

5. cost_events 컬럼 — 비용 회계의 정밀도

`cost_events` 테이블이 의미심장한 두 가지를 가진다.

- `cached_input_tokens` 별도 컬럼 — Anthropic, OpenAI, Gemini가 모두 채택한 prompt caching 가격 모델(캐시된 입력은 일반 입력 대비 90% 할인 등)을 정확히 반영하기 위함이다.
- `billing_code` 컬럼 — `cost_events` · `issues` 에 함께 달리는 자유 형식 라벨로, 외부 회계 시스템과의 매핑 키 또는 이슈 트리 묶음 식별자로 쓴다. 상위 요청자 자동 롤업 키로는 동작하지 않는다.

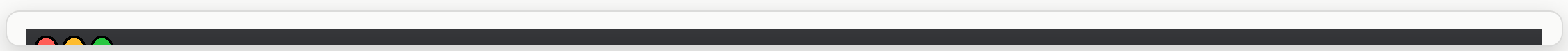
프로젝트 단위 attribution 의 실제 경로는 더 미묘하다 — `costs.byProject`

(`server/src/services/costs.ts:454~492`) 는 `coalesce(cost_events.project_id, run→activity_log→issue→project)` 순으로 프로젝트를 결정한다. 즉 (1) cost event 에 `project_id` 가 직접 달려 있으면 그것, 아니면 (2) `heartbeat_run_id` → `activity_log.run_id` → 같은 회사 issue → `issues.project_id` 로 따라간다. 가격 모델 자체의 자세한 비교는 [docs/research/10-llm-cost-budgeting.md](https://docs.research/10-llm-cost-budgeting.md).

6. activity_log — 영구 감사 흔적

모든 mutating action은 `activity_log` 한 줄을 남긴다. 실제 컬럼은 다음과 같다 (`packages/db/src/schema/activity_log.ts`). 코드 3 은 핵심 컬럼을 한 화면에 모은 것이며, `actor_type` 의 4분류 (`system / user / agent / plugin`)와 `entity_type` 의 자유 라벨이 결합해 “누가 무엇에 무엇을 했나”를 한 줄로 표현한다. schema 자체는 text 컬럼이고 허용 집합은 `service/validator`가 정의한다.

코드 3. `activity_log` 테이블의 핵심 컬럼



이 테이블은 두 가지 가치를 가진다. (1) **감사** — 누가 언제 무엇을 바꿨나가 길게 보존된다. (2) **에이전트가 자기 활동을 회상** — 자기 회사의 최근 활동 피드를 조회해 컨텍스트를 잡을 수 있다. 추가로 `run_id` 가 cost attribution의 다리 역할을 한다 (§ 5 참고). 자동 복구·watchdog 경로의 `recovery/reconciliation` 서비스도 `actor_type='system'` 으로 `issue.updated`, `heartbeat.output_stale_escalated`, `issue.blockers.updated`, `issue.harness_liveness_escalation_created` 같은 활동을 남기며, `details.source` 또는 연결된 `issue_recovery_actions.id` 가 자동 복구 경로의 감사 단서가 된다.

7. 정기 작업 — `routines` 테이블

매일 / 매주 / 매시간 도는 작업 — 고객 지원 트리아지, 마케팅 리뷰, 메트릭 수집 — 은 `routines` 에 등록된다. cron 표현식 + 어떤 에이전트의 어떤 heartbeat가 깨어나야 하는지를 정의하며, 보드 UI의 `/routines` 페이지에서 관리된다.

*“recurring jobs you have to remember to manually kick off”*라는 README 문제 정의의 직접적 해법이다.

거버넌스 관점에서 `routines` 는 단순한 cron 래퍼가 아니다. 사람이 매번 누르지 않아도 회사가 움직이게 만드는 장치이면서, 동시에 어떤 반복 작업이 어떤 에이전트를 깨우는지 보드가 사전에 승인·수정할 수 있는 통제 표면이다. 반복 작업이 비용 폭주와 가장 쉽게 결합되는 지점이므로, routine은 budget policy와 함께 읽어야 한다. 예를 들어 “매시간 경쟁사 가격 조사” routine은 정상일 때는 운영 자동화지만, 대상 사이트가 느려지거나 모델 호출이 길어지면 매시간 hard cap을 향해 비용을 누적시키는 루프가 된다. 그래서 routine의 설계 품질은 cron 표현식보다 `agent_id`, 예상 비용, 실패 시 재시도 정책, 일시정지 권한이 한 화면에서 보이는지에 달려 있다.

8. 거버넌스 의도의 한 줄 압축

Paperclip 은 자동화하지 않는다 — 자동화를 선언 하고

감시

한다. 보드는 평소 잠자코 있어도 되지만, hard ceiling, 승인 큐, 활동 로그, 일시정지 권한이 같은 자리에 항상 있다.

V1 범위 밖이지만 ROADMAP 에 들어 있는 것 — 멀티 멤버 보드, 자동 회사 stop-loss, 매출 회계 — 도 모두 이 거버넌스 축의 확장이지 별개의 시스템이 아니다.

[08-usage.md](#)는 위 메커니즘이 실제 운영 흐름에서 어떻게 연결되는지 — 설치 · 회사 만들기 · 첫 에이전트 · CEO 전략 승인 · 비용 모니터링 — 를 시간 순서로 분석한다.

Usage — 설치 · 회사 만들기 · 첫 운영

1. 사전 준비

요구	버전	비고
Node.js	≥ 20 (LTS)	<code>package.json engines</code> 강제
pnpm	≥ 9	<code>packageManager: pnpm@9.15.4</code>
OS	macOS · Linux · Windows	embedded-postgres 18 native 바이너리가 깔린다
(옵션) Docker	24+	local Docker Postgres 모드용
(옵션) Tailscale	—	private 네트워크 모드용

2. 5분 안에 시작하기 — `local_trusted` 모드

가장 마찰이 적은 방법은 임베디드 Postgres + 로그인 없는 로컬 모드다. **코드 1**은 4줄로 끝나는 시작 절차다 — Node 20+, pnpm 9+ 외에 추가 설치하는 없으며, `pnpm dev`가 임베디드 PostgreSQL · 마이그레이션 · API · UI를 한 origin 위에 띄운다.

코드 1. `local_trusted` 모드 — 5분 시작 시퀀스



`pnpm dev`가 만든 부산물은 `~/.paperclip/instances/default/` 아래에 산다. 이 모드에서는 보드 UI가 즉시 회사 생성 화면으로 데려간다. 인증·세션은 없으며, 단일 운영자(`local-board` pseudo-user)가 모든 권한을 가진다.

리셋이 필요하면 `~/.paperclip/instances/default/db/` 디렉터리만 지우고 다시 띄우면 된다.

3. CLI 기본 흐름 — `paperclipai`

`pnpm paperclipai` 명령은 `cli/src/index.ts` 의 디스패처를 부른다. 자주 쓰는 서브커맨드는 **코드 2**와 같다 — 운영 라이프사이클(`onboard` → `run` → `doctor`)은 위쪽, 부수 작업(`configure` · `plugin install` · `db:backup`)은 아래쪽이다.

코드 2. `paperclipai` CLI — 자주 쓰는 서브커맨드



```
onboard
```

`onboard` 가 묻는 첫 질문은 reachability다(자세한 내용 `doc/DEPLOYMENT-MODES.md` § 4). **코드 3**은 4가지 reachability 옵션이며, 1번이 *나만 쓴다*, 2\~3번이 *팀이 쓴다*, 4번이 *외부에 공개한다*는 운영 톤에 대응한다.

코드 3. `onboard` reachability 4가지 옵션

4. 보드 UI 첫 화면 — 회사를 만들자

UI가 처음 띄우는 흐름은 두 갈래다.

(A) **새 회사 만들기** - `Company name` — 첫 화면 필수 입력 - `Mission / goal` — 선택, 비워 둘 수 있음 (e.g., “*Build the #1 AI note-taking app to \$1M MRR.*”)

첫 화면은 이 두 입력으로 끝난다(`ui/src/components/OnboardingWizard.tsx:681,700`). `description` · `brand_color` · `attachment_max_bytes` 같은 항목은 회사 설정(`/company/settings`)에서, **월 예산**은 별도의 예산 API(`PATCH /companies/:companyId/budgets`, `server/src/routes/costs.ts:280`)와 비용 화면(`/costs`)에서 조정한다. `issue_prefix` 는 서버가 회사명에서 자동 파생한다(`server/src/services/companies.ts:140-149`).

(B) **기존 회사 불러오기 (import)** - `/company/import` 페이지에서 회사 패키지 (Agent Companies spec/v1 호환) 업로드 - 템플릿 모드(구조만) 또는 스냅샷 모드(상태 포함) 선택

회사가 만들어지면 사이드바에 회사 항목이 추가되고, 모든 후속 작업은 그 회사 컨텍스트 안에서 일어난다.

5. 첫 에이전트 — CEO 부터

`/agents/new` 페이지가 다음을 묻는다.

입력	예시	설명
name	“Aurora” (첫 에이전트는 자동 <code>CEO</code>)	사람 친화 이름
title	CEO	표시용
role	<code>ceo</code> (첫 에이전트 자동) / <code>engineer</code> / <code>marketer</code> ...	첫 에이전트는 자동으로 <code>ceo</code> , 이후 자유 선택
reports_to	(없음 — CEO 만 NULL)	두 번째부터 picker 활성화
adapter_type	<code>claude_local</code>	어떤 런타임으로 깰까
adapter_config	<code>jsonb</code>	어댑터마다 다른 모양 (Claude Code 면 SKILL/CLAUDE.md, system prompt, model, max_turns 등)
AI Odyssey runtime_config	<code>jsonb</code>	heartbeat · interval · cheap model 등 (<code>buildNewAgentRuntimeConfig</code>)

6. 첫 골과 첫 이슈

`/projects` 또는 `/goals` 에서 회사 골을 분해한다. SPEC § 5의 골 위계는 *Initiative* → *Project* → *Milestone* → *Issue* → *Sub-issue*다. 실용적으로는 다음 패턴이 유효하다.

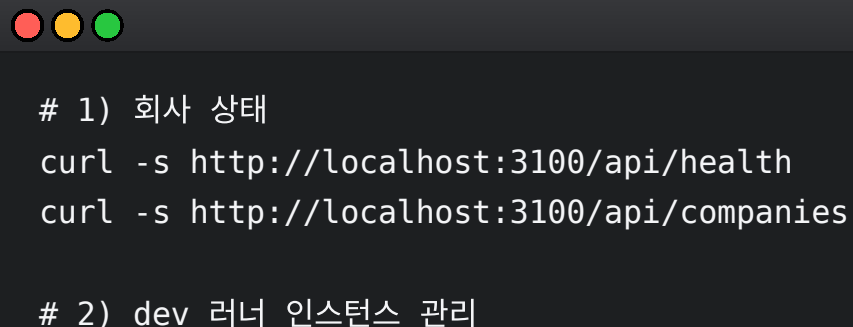
1. CEO가 첫 heartbeat에서 골을 받고 *전략 분해*를 만든다.
2. 보드는 그 분해를 `/approvals` 에서 승인한다.
3. CEO의 다음 heartbeat가 자기 보고 에이전트들에게 이슈를 만들어 할당한다.
4. 자식 에이전트가 자기 차례에 일을 하고, 비용을 보고하고, 결과를 `in_review` 로 핸드오프한다.

이슈 ID는 `companies.issue_prefix` + `issue_counter` 로 회사별 단조 증가 — 예: `PAP-1`, `PAP-2`, ...

7. 운영 중 흔한 점검 명령

코드 4는 일상 운영의 5가지 점검 묶음이다 — 회사 상태 → 러너 인스턴스 → 마이그레이션 → 전체 검증 → E2E의 사다리. 위에서 아래로 갈수록 무거운 검증이며, 일상은 1\~2번까지로 충분하고 PR 직전에는 4번까지 돌린다.

코드 4. 일상 점검 5묶음 — health · runner · migration · validation · E2E



```
# 1) 회사 상태
curl -s http://localhost:3100/api/health
curl -s http://localhost:3100/api/companies

# 2) dev 러너 인스턴스 관리
```

`AGENTS.md` § 7의 권고는 “PR 핸드오프 전에는 전체 체크, 그 외에는 가장 좁은 검증”이다.

8. Docker 한 줄로 띄우기 (옵션)

코드 5는 두 가지 Docker 사용 패턴이다 — `docker compose` 로 **PostgreSQL 17 + Paperclip server** 풀스택을 한 명령으로 띄우거나, `docker build` + `docker run` 으로 호스트 DB(또는 임베디드 Postgres) 위에 Paperclip 컨테이너만 띄우거나. 후자가 *완전 portable*이라 운영 이전이 가장 단순.

코드 5. Docker 사용 패턴 — full-stack compose vs Paperclip 단독 컨테이너



위 compose 는 `db` (postgres:17-alpine) 와 `server` (Paperclip Dockerfile) 두 서비스를 함께 띄우며 `BETTER_AUTH_SECRET` 이 강제된다(`docker/docker-compose.yml:18-31` , `doc/DOCKER.md:84-89`). 단독 컨테이너 경로도 마찬가지로 `BETTER_AUTH_SECRET` 이 필요한데, 루트 `Dockerfile` 의 기본값이 `PAPERCLIP_DEPLOYMENT_MODE=authenticated` (`Dockerfile:76`)이기 때문이다. 단독 컨테이너는 임베디드 Postgres 모드를 그대로 쓰므로 `/paperclip` 볼륨만 보존하면 된다(`doc/DOCKER.md`).

9. 호스팅 시나리오 — Supabase + Vercel

규모가 커지면 호스티드로 옮긴다(`doc/DATABASE.md` § 3). 코드 6 은 Supabase + Vercel 같은 매니지드 환경으로 옮길 때 필요한 핵심 환경 변수다 — DB/마이그레이션·secrets·auth 세 묶음으로 나뉜다.

코드 6. 호스팅 시나리오 핵심 환경 변수 (Supabase + Vercel 예시)



```
authenticated/public
```

 모드에서는 Better Auth 세션이 강제되며 `BETTER_AUTH_SECRET` 과 `auth.publicBaseUrl` 이 함께 필수다(`cli/src/checks/deployment-auth-check.ts:28-60`). env-only 호스팅을 만들려면 `PAPERCLIP_DEPLOYMENT_MODE` / `EXPOSURE` 도 명시해야 한다 — 미설정이면 `local_trusted` + private 으로 떨어진다(`server/src/config.ts:160-180`).

`local_trusted` 로 오래 사용하다 `authenticated` 로 전환하는 경우 — instance admin 이 `local-board` 한 명뿐일 때에 한해 — 부팅 시 1회용 board-claim URL 이 경고로 표시되어, 로그인한 사용자가 instance admin 으로 승격되고 `local-board` 권한이 강등된다(`doc/DEPLOYMENT-MODES.md:115-126` , `server/src/board-claim.ts:43-66`).

10. 초기 운영 체크포인트

- **예산 한도** 는 보수적으로 설정하고 `unlimited` 사용을 제한한다. 초기 폭주 비용이 가장 흔한 운영 리스크다.
- **승인 게이트** (`require_board_approval_for_new_agents=true`)를 켜 두면 CEO가 만들려는 에이전트를 보드가 먼저 검토할 수 있다.
- `/dashboard/live` 는 회사 WebSocket(`/api/companies/:companyId/events/ws`) 위에 *활성/최근 에이전트 run* 을 실시간으로 보여 준다 (`ui/src/pages/DashboardLive.tsx:42-46`). 회사 *활동 로그* (`/activity`)와는 다른 화면이다.
- `/approvals` 큐와 **활동 로그** 는 일상 점검의 최소 표면이다.

[09-pros-cons.md](#)는 이 모든 메커니즘의 *강점·약점·트레이드오프*를 정리한다.

Pros & Cons — Paperclip의 트레이드오프

1. 평가 프레임

이 장은 한 가지 질문에 답한다 — *언제 Paperclip을 쓰고, 언제 쓰지 않아야 하는가?* 평가는 다음 6개 축에서 이뤄진다.

1. **개념적 명료성** — 무엇을 푸는지가 명확한가
2. **데이터 모델의 견고성** — 작업·비용·org가 일관되게 모델링됐는가
3. **운영 안전망** — 자동화 대비 인간 통제의 균형
4. **확장성** — 새 어댑터·플러그인을 코어 수정 없이 추가 가능한가
5. **운영 부담** — 설치·업데이트·모니터링의 무게
6. **성숙도** — 누락된 영역과 향후 리스크

2. 강점

표 1 Paperclip의 강점

강점	코드/문서 근거	왜 중요한가
명확한 책임 분리 — control plane vs execution plane	장기 SPEC <code>invoke/status/cancel</code> (<code>doc/SPEC.md:207-215</code>) vs 현재 구현 <code>execute</code> + <code>testEnvironment</code> 필수(<code>packages/adapter-utils/src/types.ts:349-352</code>) — 어댑터 계약이 두 층으로 얹게 유지된다	새 런타임이 등장해도(예: GPT-5 native CLI) 어댑터만 추가하면 된다. 코어가 에이전트 자체를 흉내내지 않는다는 점 장기 유지 부담을 줄인다.
회사를 1급 객체로	80여 개 테이블 중 60여 개가 <code>company_id</code> 컬럼 보유, multi-company 격리	한 인스턴스로 여러 자율 사업체를 운영 시나리오가 자연스럽다. 데이터 격리가 강제되어 보안 회귀가 적다.
단일 assignee + atomic checkout	<code>issues.checkout_run_id</code> 를 중심으로 한 조건부 UPDATE (<code>server/src/services/issues.ts:4659-4809</code> — 의사코드는 02장 § 5)	동시성 문제를 데이터 모델 차원에서 CRDT/낙관적 락 없이 다중 조건 UPDATE 한 번으로 lock + status + adopt.
보수적 회복 모델	<code>doc/execution-semantics.md:407-422</code> § 12 + <code>doc/SPEC-implementation.md:434-442</code> V1 liveness rule — “<i>preserve ownership / retry once</i>”	자동 재할당의 위험(잘못된 할당, 중복)을 거부하고, 명시적인 회복 이슈로 보

3. 약점 / 미성숙 영역

표 2 약점과 그 원인

약점	어디서 드러나는가	영향
거대 단일 파일들	<code>routes/issues.ts</code> 205 KB, <code>services/issues.ts</code> 201 KB, <code>services/company-portability.ts</code> 178 KB, <code>ui/pages/IssueDetail.tsx</code> 163 KB, <code>routes/access.ts</code> 140 KB, <code>routes/agents.ts</code> 119 KB, <code>routes/plugins.ts</code> 93 KB	가독성·테스트성 저해. 신규 기여자가 한 번에 구조를 파악하기 어렵다. 서비스 분할이 점진적 과제.
자동 재할당 부재	execution-semantics § 12의 “ <i>does not automatically reassign</i> ”	에이전트 한 명이 사라지면 보드가 직접 처리해야 한다. 본질적으로 <i>의도된</i> 약점이지만 실수로 사람이 떠난 경우 시스템이 그걸 <i>알려 줄 뿐 해결</i> 하지 않는다.
고급 거버넌스 부재 (v1)	회사 멤버십(<code>packages/db/src/schema/company_memberships.ts</code>)과 4 단계 휴먼 role(owner/admin/operator/viewer, <code>packages/shared/src/constants.ts:624-630</code>)은 이미 있으나, <code>approvalsNeeded > 1</code> 다중 승인이 거부되고	4-eye 결재·승인 위임·다중 멤버 보드를 <i>코어</i> 에서 <i>88 / 98</i> 표현하기 어렵다.

4. 트레이드오프 — 의도된 절충

다음은 약점이 아니라 *결정* 이다.

표 3 의도된 트레이드오프

결정	대안	Paperclip의 선택 이유
단일 assignee · 다중 조건 atomic checkout	다수 assignee · 락 free	단순성. 동시성 분쟁이 데이터 모델 + 한 번의 조건부 UPDATE 로 사라진다.
자동 회복은 한 번만	무한 재시도 또는 다른 에이전트로 재할당	잘못된 자동화의 위험(잘못된 오너, 중복 비용) 회피. 모호하면 사람에게 드러낸다.
Tasks + comments 만, 별도 채팅 X	별도 메신저(Slack-like)	모든 컨텍스트가 작업에 부착되어 감사 일치성 확보. 인박스 파편화 회피.
process / http 같은 범용 어댑터 보존	풍부한 어댑터만	“호출 가능한 것은 모두 채용된다”는 약속을 깨지 않기 위해.
멀티-회사 단일 인스턴스	회사당 인스턴스	운영 부담 절감. 인스턴스를 하나만 돌보면 된다.
Drizzle 스키마-퍼스트	Prisma 같은 schema DSL	TypeScript-native, SQL-like, 마이그레이션 미세 제어.

5. 누구에게 적합한가

적합:

- 한 사람 / 작은 팀이 자율 에이전트 *회사*를 운영하고 싶은 경우
- 여러 코딩 에이전트(Claude Code · Codex · Cursor 등)를 통합 거버넌스로 다루고 싶은 경우
- 토큰 비용 통제가 비즈니스 임계점인 경우
- 자기-호스팅을 받아들이는 경우 (SaaS 이용 의지 X)

부적합:

- *지금 당장* 매출/비용 통합 회계가 필요한 경우 (플러그인을 직접 만들거나 외부 도구 사용)
- 대규모 KB / RAG 가 핵심 가치인 경우 (Paperclip 코어가 이를 다루지 않음)
- 풀 SaaS 매니지드를 원하는 경우 (V1 은 self-hosted only)

- 멀티 멤버 보드 / 4-eye 결재가 즉시 필요한 엔터프라이즈

6. 성숙도 평가 — 한 줄 요약

Paperclip은 자율 에이전트 회사를 위한

최소 충분

control plane이다. 핵심 도메인(이슈 · 비용 · 거버넌스 · 어댑터 · 플러그인)이 견고히 모델링됐고,

자동화하지 않는 결정

들이 일관되게 적용되어 있다. 큰 단일 파일들과 KB/매출 영역의 부재가 향후 작업이지만, 학습용 도구이자 첫 자율 회사를 운영하기 위한 출발점으로 충분하다.

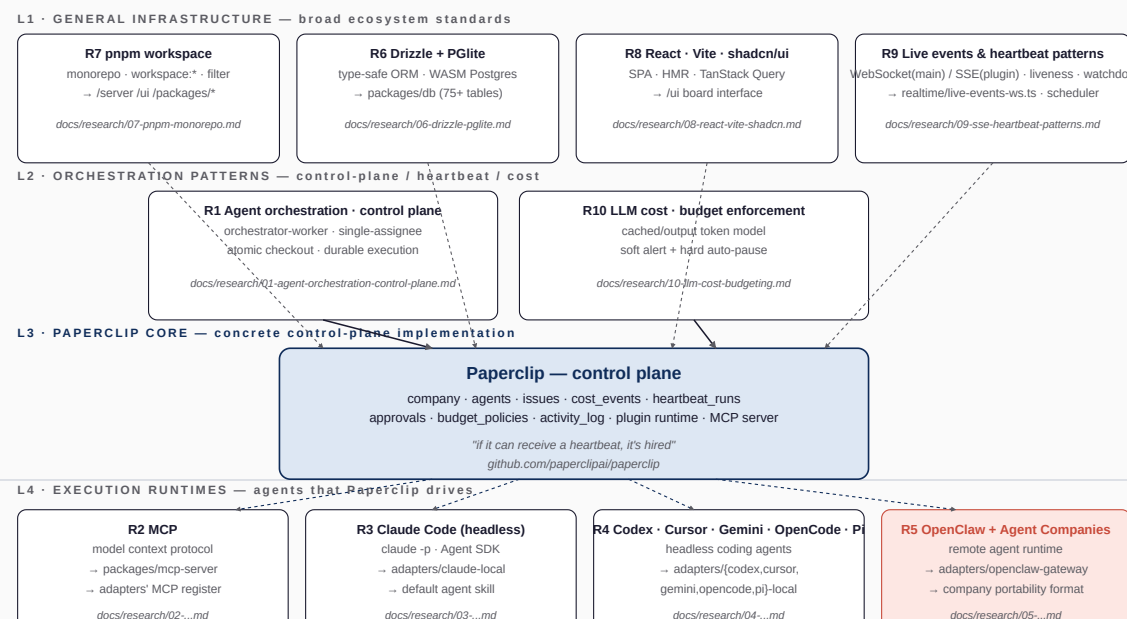
[10-research-map.md](#)는 10편의 리서치가 Paperclip의 어떤 모듈을 떠받치는지 매핑한다.

Research Map — 10편의 리서치로 본 Paperclip의 기술적 좌표

1. 4-층 위계

리서치 10편은 Paperclip 본체를 둘러싼 **4개의 동심층** 으로 정렬할 수 있다 — 가장 바깥의 **일반 인프라 표준** (R6\~R9) → **오케스트레이션 패턴** (R1, R10) → **Paperclip 코어** (이 리포트의 본체) → **실행 런타임** (R2\~R5). 그림 10이 그 위계를 한 페이지로 보여 준다.

그림 10. 리서치 ↔ Paperclip 4-층 위계 — 일반 인프라 표준 / 오케스트레이션 패턴 / Paperclip 코어 / 실행 런타임



2. 리서치 ↔ Paperclip 모듈 일대일 매핑

표 1 리서치 ↔ Paperclip 매핑

#	리서치	다루는 영역	Paperclip의 대응 위치
R1	Agent orchestration / control plane	orchestrator-worker · single-assignee · atomic checkout · durable execution	SPEC § 8 atomic checkout, services/recovery/*, doc/execution-semantics.md
R2	Model Context Protocol	host/client/server · transport · tools/resources/prompts	packages/mcp-server, adapters/*-local의 MCP 등록
R3	Claude Code (headless)	<code>claude -p</code> · stream-json · Agent SDK · skills	packages/adapters/claude-local
R4	Codex · Cursor · Gemini · OpenCode · Pi	헤드리스 코딩 에이전트의 공통 계약	packages/adapters/{codex,cursor,gemini,opencode,pi}-local
R5	OpenClaw + Agent	원격 에이전트 런타임 · 회사 패키지 프레임	packages/adapters/openclaw-gateway, services/company-availability (120 KB)

3. 외부 표준 다이어그램으로 본 Paperclip의 좌표

3.1 Anthropic의 4가지 패턴

Anthropic의 “*Building Effective Agents*” (Schluntz & Zhang)가 정의한 4가지 빌딩 블록이 Paperclip 어디에 대응되는지 본다.

그림 10-1 (Augmented LLM) 은 도구·리트리벌·메모리를 가진 단일 LLM 이다. Paperclip의 *한 명의 에이전트 안에서* 일어나는 일이며, Paperclip 자체는 이 블록을 정의하지 않고 어댑터에 위임한다.

그림 10-1. Augmented LLM (출처: Anthropic — Building Effective Agents)



4. 어디까지가 *Paperclip*의 발명인가

위 매핑이 보여 주는 핵심 통찰은 다음과 같다 — Paperclip은 기술보다는 *정합성*의 발명이다.

- **데이터 모델로 동시성을 푼다** — atomic checkout, single assignee, monthly budget 윈도우.
- **자동화의 한계를 명시한다** — “*Paperclip reports problems, it doesn’t silently fix them*” 한 줄로 회복 정책의 모든 모서리가 결정된다.
- **회사를 1급 객체로** — 비용·거버넌스·작업이 모두 같은 스코프에서 흐르므로, 별도 BI 도구 없이도 “*회사가 돌고 있나*” 라는 질문에 답할 수 있다.
- **어댑터 계약의 단순함** — 장기 SPEC 의 `invoke/status/cancel` 3-method 계약(`doc/SPEC.md:207-215`) + 현재 구현의 `ServerAdapterModule.execute` / `testEnvironment` 2 필수 메서드(`packages/adapters/claude-local/package.json:15-20`) + `utils/src/types.ts:349-431`) + adapter package 의 4 진입점(`.` / `./server` / `./ui` / `./cli` , `packages/adapters/claude-local/package.json:15-20`) 조합으로 새 런타임을 첨가하는 비용을 작게 유지한다.

5. 검증 시나리오

다음 실험은 리포트의 주요 주장을 한 사이클 안에서 확인하는 최소 시나리오다. **코드 1**은 그 *7단계 한 사이클*이다 — 클론에서 시작해 한 회사를 돌려 보고, hard limit을 일부러 트리거해 보드 거버넌스가 어떻게 동작하는지를 *체험*하는 것이 핵심.

코드 1. 검증 시나리오 — 7단계 한 사이클

```
# 1) 클론·실행
git clone https://github.com/paperclipai/paperclip.git && cd paperclip && pnpm install && pnpm dev
```

이 한 사이클은 README와 `doc/SPEC.md`, 그리고 이 리포트가 문장만으로는 제공하지 못하는 *체험*의 마지막 한 조각을 제공한다.

6. 닫는 한 줄

Paperclip 은 단일 신기술이 아니라

5\~6개 표준 도구를 한 군데에 정합적으로 묶은 것

이다. 그 정합성이 곧 가치이며, 이 리포트의 10편 리서치가 그 정합의 외부 좌표를 표시한다.